

Audio Compression Project

CODE SUBMISSION

Course: ET4262E - Multimedia Data Compression and Coding

This project implements a full audio processing pipeline:

Input Audio → Separation → Compression → Reconstruction → Evaluation

Step 1: Source Separation

File: `analyze_single.py`

Separate input WAV into 4 stems using Demucs `htdemucs` model.

Step 2: Compression

File: `compress_stems.py`

Apply dynamic range compression to each stem using Pedalboard or FFmpeg.

Step 3: Reconstruction

File: `reconstruct.py`

Overlay the 4 compressed stems back into a single mixed WAV file.

Step 4: Bitrate Analysis

File: `show_bitrate.py`

Display sample rate and calculated bitrate for each compressed stem.

Step 5: Audio Analysis

File: `analyze_single.py`

Plot waveform, RMS energy, and mel spectrogram for each stem.

1. Setup

1.1 Install system tools

```
sudo apt update
sudo apt install -y audacity ffmpeg python3 python3-pip
```

1.2 Install Python libraries

```
pip install demucs soundfile scipy torch
pip install pedalboard
pip install numpy librosa matplotlib
pip install pydub
```

Note: torchaudio is NOT required. The separation script uses soundfile for all audio I/O, bypassing the torchaudio/torchcodec save error (libnppicc.so.13).

1.3 Create project folders

```
mkdir -p output/stems
mkdir -p output/compressed
mkdir -p output/reconstructed
```

1.4 Verify FFmpeg

```
ffmpeg -version
ffmpeg -encoders | grep aac
```

2. Project Structure

```
project/
|-- analyze_single.py      # Step 1 + Step 5
|-- compress_stems.py     # Step 2
|-- reconstruct.py        # Step 3
|-- show_bitrate.py       # Step 4
|
|-- output/
|   |-- stems/
|   |   |-- vocals.wav
|   |   |-- drums.wav
|   |   |-- bass.wav
|   |   |-- other.wav
|   |
|   |-- compressed/
|   |   |-- vocalscompress.wav
|   |   |-- drumscompress.wav
|   |   |-- basscompress.wav
|   |   |-- othercompress.wav
|   |
|   |-- reconstructed/
|       |-- reconstructed_mix.wav
```

3. Python Code

3.1 analyze_single.py (Step 1 + Step 5)

Separates input WAV into 4 stems, then plots waveform and mel spectrogram.

Run: `python3 analyze_single.py nhac.wav`

```
import os
import argparse
import numpy as np
import soundfile as sf
import torch
import librosa
import librosa.display
import matplotlib.pyplot as plt

from math import gcd
from scipy.signal import resample_poly
from demucs.pretrained import get_model
from demucs.apply import apply_model

# =====
# PART 1: SOURCE SEPARATION
# =====

def load_wav_soundfile(path, target_sr):
    wav_np, orig_sr = sf.read(path, dtype="float32", always_2d=True)
    if orig_sr != target_sr:
        g = gcd(orig_sr, target_sr)
        up, down = target_sr // g, orig_sr // g
        wav_np = np.stack([
            resample_poly(wav_np[:, c], up, down)
            for c in range(wav_np.shape[1])
        ], axis=1)
    wav = torch.from_numpy(wav_np.T)
    if wav.shape[0] == 1:
        wav = wav.repeat(2, 1)
    wav = wav[:2]
    return wav

def save_stem_soundfile(tensor, path, sr):
    audio = tensor.cpu().numpy().T
    audio = np.clip(audio, -1.0, 1.0)
    sf.write(path, audio, sr, subtype="PCM_24")

def separate_stems(input_wav, output_dir="output/stems",
                  device="cpu", model_name="htdemucs"):
    if not os.path.isfile(input_wav):
        raise FileNotFoundError("File not found: " + input_wav)

    os.makedirs(output_dir, exist_ok=True)

    print("[1/3] Loading model: " + model_name)
    model = get_model(model_name)
    model.to(device)
    model.eval()

    sr = model.samplerate
    sources = list(model.sources)
```

```

print("      Sample rate : " + str(sr) + " Hz")
print("      Stems      : " + str(sources))

print("[2/3] Loading audio: " + input_wav)
wav = load_wav_soundfile(input_wav, target_sr=sr)

ref = wav.mean(0)
mean = ref.mean()
std = ref.std() + 1e-8
wav_norm = (wav - mean) / std

print("[3/3] Separating on " + device.upper() + "...")
with torch.no_grad():
    out = apply_model(
        model,
        wav_norm[None].to(device),
        device=device,
        shifts=1,
        split=True,
        overlap=0.25,
        progress=True,
    )
out = out[0] * std + mean

saved = {}
for i, stem in enumerate(sources):
    dest = os.path.join(output_dir, stem + ".wav")
    save_stem_soundfile(out[i], dest, sr)
    dur = out[i].shape[-1] / sr
    size_mb = os.path.getsize(dest) / 1024 / 1024
    print(" Saved: " + dest +
          " (" + f"{dur:.1f}s, {size_mb:.1f} MB" + ")")
    saved[stem] = dest

return saved

# =====
# PART 2: WAVEFORM + SPECTROGRAM ANALYSIS
# =====

def _plot_stems(stem_files, input_dir, title):
    """
    Internal helper: plot waveform + mel spectrogram for a list of files.
    stem_files : list of filenames to plot (must exist in input_dir)
    title      : figure title
    """
    fig, axes = plt.subplots(nrows=len(stem_files), ncols=2, figsize=(18, 15))
    fig.suptitle(title, fontsize=14)

    for i, stem_name in enumerate(stem_files):
        path = os.path.join(input_dir, stem_name)
        if not os.path.exists(path):
            print("Skip: file not found: " + path)
            continue

        y, sr = librosa.load(path, sr=None)

        # Left column: Waveform + RMS Energy
        rms = librosa.feature.rms(y=y)[0]
        times = librosa.times_like(rms, sr=sr)
        axes[i, 0].plot(np.linspace(0, len(y) / sr, len(y)), y,
                        alpha=0.5, label="Waveform")
        axes[i, 0].plot(times, rms, color="red", linewidth=2,

```

```

        label="RMS Energy")
    axes[i, 0].set_title("Dynamics: " + stem_name)
    axes[i, 0].set_xlabel("Time (s)")
    axes[i, 0].set_ylabel("Amplitude")
    axes[i, 0].legend(loc="upper right")

    # Right column: Mel Spectrogram
    S = librosa.feature.melspectrogram(y=y, sr=sr)
    S_db = librosa.power_to_db(S, ref=np.max)
    img = librosa.display.specshow(S_db, sr=sr,
                                    x_axis="time", y_axis="mel",
                                    ax=axes[i, 1])
    axes[i, 1].set_title("Spectrogram: " + stem_name)
    fig.colorbar(img, ax=axes[i, 1], format="%+2.0f dB")

plt.tight_layout()
plt.show()

def analyze_stems(input_dir="output/stems"):
    """Step 1 result: waveform + spectrogram of separated stems."""
    stems = ["vocals.wav", "drums.wav", "bass.wav", "other.wav"]
    _plot_stems(stems, input_dir,
                "Waveform and Mel Spectrogram: Separated Stems")

def analyze_compressed(compressed_dir="output/compressed"):
    """Step 2 result: waveform + spectrogram of compressed stems."""
    stems = [
        "vocalscompress.wav",
        "drumscompress.wav",
        "basscompress.wav",
        "othercompress.wav",
    ]
    _plot_stems(stems, compressed_dir,
                "Waveform and Mel Spectrogram: Compressed Stems")

# =====
# MAIN
# =====

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Separate and analyze audio stems")
    parser.add_argument("input", help="Input WAV file, e.g. nhac.wav")
    parser.add_argument("-o", "--output_dir", default="output/stems")
    parser.add_argument("-c", "--compressed_dir", default="output/compressed")
    parser.add_argument("-d", "--device", default="cpu",
                        help="cpu or cuda")
    parser.add_argument("-m", "--model", default="htdemucs")
    args = parser.parse_args()

    # Step 1: separate
    separate_stems(
        input_wav=args.input,
        output_dir=args.output_dir,
        device=args.device,
        model_name=args.model,
    )

    # Step 1 visualization: waveform + spectrogram of raw stems
    print("\nShowing analysis for separated stems...")
    analyze_stems(input_dir=args.output_dir)

```

```
# Step 2 visualization: waveform + spectrogram of compressed stems
# (runs only if output/compressed/ already contains the files)
if os.path.isdir(args.compressed_dir):
    print("\nShowing analysis for compressed stems...")
    analyze_compressed(compressed_dir=args.compressed_dir)
```

3.2 compress_stems.py (Step 2)

Applies dynamic range compression to each stem. Supports pedalboard and FFmpeg engines.

Run: `python3 compress_stems.py`

```
import os
import argparse
import subprocess
import numpy as np
import soundfile as sf
from pedalboard import Pedalboard, Compressor, Gain, Limiter
from pedalboard import HighpassFilter, LowpassFilter

# =====
# COMPRESSOR PRESETS FOR EACH STEM
#
# threshold_db : level above which compression starts
# ratio        : compression ratio (4.0 = 4:1)
# attack_ms    : time for compressor to engage (ms)
# release_ms   : time for compressor to disengage (ms)
# gain_db      : makeup gain applied after compression
# hp_hz       : high-pass filter cutoff (removes low rumble)
# =====

PRESETS = {
    "vocals": dict(
        threshold_db=-18, ratio=3.5,
        attack_ms=8.0,   release_ms=60.0,
        gain_db=3.0,     hp_hz=80.0,
    ),
    "drums": dict(
        threshold_db=-16, ratio=6.0,
        attack_ms=2.0,   release_ms=40.0,
        gain_db=5.0,     hp_hz=40.0,
    ),
    "bass": dict(
        threshold_db=-20, ratio=4.0,
        attack_ms=10.0,  release_ms=100.0,
        gain_db=4.0,     hp_hz=30.0,
    ),
    "other": dict(
        threshold_db=-22, ratio=2.5,
        attack_ms=20.0,  release_ms=120.0,
        gain_db=2.0,     hp_hz=60.0,
    ),
}

# =====
# METHOD A: PEDALBOARD (default)
# =====

def compress_pedalboard(wav_np, sr, stem):
    """
    Apply effect chain: HPF -> Compressor -> Makeup Gain -> Limiter
    wav_np : (samples, channels) float32
    returns: (samples, channels) float32
    """
    p = PRESETS[stem]

    chain = []
    if p["hp_hz"] > 0:
        chain.append(HighpassFilter(cutoff_frequency_hz=p["hp_hz"]))
```



```

chain.append(Compressor(
    threshold_db=p["threshold_db"],
    ratio=p["ratio"],
    attack_ms=p["attack_ms"],
    release_ms=p["release_ms"],
))
chain.append(Gain(gain_db=p["gain_db"]))
chain.append(Limiter(threshold_db=-1.0, release_ms=100.0))

board = Pedalboard(chain)
wav_ch = wav_np.T.astype(np.float32) # (channels, samples)
out_ch = board(wav_ch, sample_rate=sr)
return out_ch.T # (samples, channels)

# =====
# METHOD B: FFMPEG
# =====

FFMPEG_PRESETS = {
    "vocals": "threshold=0.125:ratio=3.5:attack=8:release=60:makeup=1.4",
    "drums": "threshold=0.157:ratio=6.0:attack=2:release=40:makeup=1.78",
    "bass": "threshold=0.1:ratio=4.0:attack=10:release=100:makeup=1.58",
    "other": "threshold=0.08:ratio=2.5:attack=20:release=120:makeup=1.26",
}

def compress_ffmpeg(src, dest, stem):
    """Apply Ffmpeg acompressor filter. Output: 16-bit PCM WAV."""
    p = PRESETS[stem]
    hp = f"highpass=f={int(p['hp_hz'])}, " if p["hp_hz"] > 0 else ""
    af = hp + "acompressor=" + FFMPEG_PRESETS[stem] + ", alimiter=limit=0.891"

    cmd = [
        "ffmpeg", "-y", "-i", src,
        "-af", af,
        "-ar", "44100",
        "-acodec", "pcm_s16le",
        dest,
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode != 0:
        raise RuntimeError("FFmpeg error:\n" + result.stderr)

# =====
# MAIN PIPELINE
# =====

STEM_MAP = {
    "vocals": "vocalscompress.wav",
    "drums": "drumscompress.wav",
    "bass": "basscompress.wav",
    "other": "othercompress.wav",
}

def run_compression(input_dir="output/stems", output_dir="output/compressed",
                    engine="pedalboard", bit_depth=24):
    os.makedirs(output_dir, exist_ok=True)

    subtype_map = {16: "PCM_16", 24: "PCM_24", 32: "PCM_32"}
    subtype = subtype_map.get(bit_depth, "PCM_24")
    saved = {}

```

```

print("Engine      : " + engine.upper())
print("Bit depth : " + str(bit_depth) + "-bit PCM")
print("Input      : " + input_dir + "/")
print("Output     : " + output_dir + "/")
print("")

for stem, out_name in STEM_MAP.items():
    src = os.path.join(input_dir, stem + ".wav")
    dest = os.path.join(output_dir, out_name)

    if not os.path.isfile(src):
        print(" Skip " + stem + ": not found " + src)
        continue

    p = PRESETS[stem]
    print(" " + stem + " threshold=" + str(p["threshold_db"]) +
          "dB ratio=" + str(p["ratio"]) + ":1 " +
          "attack=" + str(p["attack_ms"]) + "ms " +
          "release=" + str(p["release_ms"]) + "ms")

    if engine == "ffmpeg":
        compress_ffmpeg(src, dest, stem)
    else:
        wav_np, sr = sf.read(src, dtype="float32", always_2d=True)
        compressed = compress_pedalboard(wav_np, sr, stem)
        sf.write(dest, compressed, sr, subtype=subtype)

    size_mb = os.path.getsize(dest) / 1024 / 1024
    print(" Saved: " + out_name + " (" + f"{size_mb:.1f} MB" + ")")
    saved[stem] = dest

return saved

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Step 2: Compress stems")
    parser.add_argument("-i", "--input_dir", default="output/stems")
    parser.add_argument("-o", "--output_dir", default="output/compressed")
    parser.add_argument("--engine", default="pedalboard",
                        choices=["pedalboard", "ffmpeg"])
    parser.add_argument("--bit_depth", default=24, type=int,
                        choices=[16, 24, 32])
    args = parser.parse_args()

    run_compression(
        input_dir=args.input_dir,
        output_dir=args.output_dir,
        engine=args.engine,
        bit_depth=args.bit_depth,
    )

```

3.3 reconstruct.py (Step 3)

Overlays the 4 compressed stems into a single reconstructed_mix.wav file.

Run: `python3 reconstruct.py`

```
from pydub import AudioSegment
import os

input_dir = "output/compressed"
output_dir = "output/reconstructed"

stems = [
    "vocalscompress.wav",
    "drumscompress.wav",
    "basscompress.wav",
    "othercompress.wav",
]

os.makedirs(output_dir, exist_ok=True)

combined = None

print("--- Starting Reconstruct Mix ---")

for stem in stems:
    path = os.path.join(input_dir, stem)
    if os.path.exists(path):
        print("Mixing: " + stem)
        audio = AudioSegment.from_wav(path)
        if combined is None:
            combined = audio
        else:
            combined = combined.overlay(audio)
    else:
        print("Warning: file not found - " + path)

output_path = os.path.join(output_dir, "reconstructed_mix.wav")

if combined is not None:
    combined.export(output_path, format="wav")
    print("")
    print("--- SUCCESS ---")
    print("Output file: " + output_path)
else:
    print("Failed: no files found to mix.")
```

3.4 show_bitrate.py (Step 4)

Uses ffprobe to read sample rate, then calculates bitrate for each compressed stem.

Run: `python3 show_bitrate.py`

```
import subprocess
import os

input_dir = "output/compressed"

stems = [
    "vocalscompress.wav",
    "drumscompress.wav",
    "basscompress.wav",
    "othercompress.wav",
]

print(f"{'File Name':<25} | {'Bitrate':<12} | {'Sample Rate':<12}")
print("-" * 55)

for stem in stems:
    path = os.path.join(input_dir, stem)
    if os.path.exists(path):
        cmd = (
            "ffprobe -v error -select_streams a:0"
            " -show_entries stream=sample_rate"
            " -of default=noprint_wrappers=1:nokey=1 "
            + path
        )
        result = subprocess.check_output(cmd, shell=True).decode().strip()
        sample_rate = result
        bitrate_kbps = (int(sample_rate) * 24 * 2) // 1000
        print(f"{stem:<25} | {bitrate_kbps:>6} kbps | {sample_rate:>10} Hz")
    else:
        print(f"{stem:<25} | File not found: {path}")
```

4. Run Summary

Execute the scripts in the following order from the project root directory:

```
# Step 1 + Step 5: Separate audio and show analysis plots
python3 analyze_single.py nhac.wav
```

```
# Step 2: Compress the separated stems
python3 compress_stems.py
```

```
# Step 3: Reconstruct a mixed file from compressed stems
python3 reconstruct.py
```

```
# Step 4: Display bitrate of each compressed stem
python3 show_bitrate.py
```

Optional flags

```
# Use GPU for faster separation
python3 analyze_single.py nhac.wav -d cuda
```

```
# Use FFmpeg engine for compression + 16-bit output
python3 compress_stems.py --engine ffmpeg --bit_depth 16
```

```
# Custom output folder
python3 analyze_single.py nhac.wav -o my_stems/
```